

Eksiksiz Ders Notu Arsivi

Tum PDF slaytlarindan cikarilmis haftalik notlar — kapsulleme, kalitim, migration, Devise dahil hicbir konu atlanmadi. 7 gunluk plan ve ezber listesi ile sinava hazirlan.

Konu ara: kapsulleme, kalitim, migration, devise..

Tum panelleri ac

Tum panelleri kapat

7 Gun Kala — Sifirdan Gecer Not Plani

Ruby/Rails bilgin yoksa bu plani takip et. Her gun 2-3 saat yeterli. Gorevleri tiklayarak ilerlemeni kaydet.

0 / 39 gorev tamamlandi (0%)

Bugunun gorevine git

1

Gun 1: Rails nedir? Temel iskelet

+

2-3 saat

MVC ve Ruby mantigini anla. Kod yazmana gerek yok.

- ☐ Hafta 2 — Basit Anlatim kutusunu oku
[Konuya git →](#)
- ☐ MVC akisini ezberle: routes → controller → model → view
[Konuya git →](#)
- ☐ Ruby Temelleri panelini ac (Integer, String, Symbol farki)
[Konuya git →](#)
- ☐ Rails Dizin Yapisini gozden gecir (app/, config/, db/)
[Konuya git →](#)
- ☐ Test: Hafta 2 sorulari (1-5) — cevaplari kontrol et
[Konuya git →](#)

Bu gunun sonunda bilmen gerekenler:

- Ruby'de her sey nesnedir
- Symbol tekil, String her seferinde yeni nesne
- ? metot → true/false, ! metot → orijinali degistirir

2**Gun 2: Veritabani & Active Record**

+

2-3 saat

ORM ve find/find_by/where farkini bil.

3**Gun 3: Routes, HTTP & CRUD**

+

2-3 saat

HTTP metotlari ve 7 CRUD route'u bil.

4**Gun 4: Formlar & Guvenlik**

+

2 saat

Strong Parameters ve form mantigini anla.

5**Gun 5: Bootstrap & Model Iliskileri**

+

2-3 saat

Grid sistemi ve tablo iliskilerini bil.

6**Gun 6: Validation, I18n & Devise**

+

2-3 saat

Dogrulama, coklu dil ve kullanici girisi konularini bitir.

7**Gun 7: Tekrar & Sinav Gunu**

+

3-4 saat

Tum test sorularini 2. kez coz. Ezber listesini tekrar et.

Konu İndeksi — Alfabetik

Tüm PDF slaytlarından çıkarılmış konu listesi. **Kapsülleme, kalitim (miras), polimorfizm** ve diğer OOP kavramları aşağıda tabloda vurgulanmıştır.

Konu	Hafta	Açıklama
Active Record	3	ORM kütüphanesi, model katmanı
Active Storage	8	Dosya/resim yükleme
Array (Dizi)	2	Ruby veri türü, metotları
attr_accessor / reader / writer	2	Kapsülleme — getter/setter
Authentication	10	Kimlik doğrulama (Devise)
Authorization	10	Yetkilendirme (admin?, roller)
before_action	5	Controller callback (DRY)
belongs_to	7	Model ilişkisi, FK bu tabloda
Bloklar (each, map, yield)	2	Ruby blok yapısı
Bootstrap Grid	6	12 sütun responsive layout
button_to	5	DELETE form butonu
Constants (Sabitler)	2	PI = 3.14, büyük harf
CRUD	4	Create Read Update Delete
CSRF / authenticity_token	5	Form güvenlik tokeni
Enum	4	Sayısal sabitlere isim (draft: 0)
ERB	4	Embedded Ruby, <% %> <%= %>
FriendlyId	9	SEO slug URL
Global değişken (\$)	2	\$monday — her yerden erişim
Hash	2	Key-value Ruby yapısı
has_many	7	Model ilişkisi

Konu	Hafta	Açıklama
has_many :through	7	Ara tablo ile ilişki
has_one	7	Bire-bir ilişki
HABTM	7	Çoktan çoğa, join table
HTTP Metotları	4	GET POST PUT PATCH DELETE
l18n	9	Çoklu dil (t, l, locale)
Instance variable (@)	2	Nesneye ait değişken
Kalıtım (Miras)	2	class Dog < Animal, super
Kapsülleme	2	@name, getter/setter, attr_*
Lambda vs Proc	2	Argüman toleransı farkı
Migration	3	Veritabanı şema değişikliği
Modül (Module)	2	include, namespacing
MVC	2	Model View Controller
Normalization	8	Veriyi kaydetmeden düzenleme
ORM	3	Tablo ↔ Sınıf eşlemesi
Partial	5	_form.html.erb yeniden kullanım
Polimorfizm	2	speak metodunu override etme
private / protected / public	2	Erişim kontrolü
Strong Parameters	5	params.require.permit
Symbol	2	:user tekil nesne
Validation	8	validates kuralları
Class variable (@@)	2	Sınıfa ait değişken

OOP Kavramları (Hafta 2 — Mutlaka Bil)

Kavram	Ruby'de Nasıl?	Örnek
Kapsülleme	Instance variable @ ile gizle, getter/setter ile eriş	@name, attr_accessor :name

Kavram	Ruby'de Nasıl?	Örnek
Kalıtım (Miras)	Alt sınıf üst sınıftan türetilir	<code>class Dog < Animal</code>
Polimorfizm	Alt sınıf metodu override eder	<code>def speak; "I'm a dog"; end</code>
super	Üst sınıf metodunu çağırır	<code>super + " from Cat class"</code>
Abstraction	Modül/sınıf ile arayüz tanımlama	<code>module Movable</code>

Sınav Ezber Listesi — 15 Kritik Madde

7. gün ve sınav sabahı bu listeyi 2 kez oku. Çogu sınav sorusu bu maddelerden türetilir.

Ruby'de her şey nesnedir

3.class → Integer → Numeric → Object

MVC akışı

routes.rb → Controller → Model → View → HTML

ORM

Veritabanı tablosu = Ruby model sınıfı

find(1)

Bulamazsa exception (hata) fırlatır

find_by(...)

Bulamazsa nil döner

where(...)

Filtreler, liste (Relation) döner

resources :posts

7 CRUD route otomatik oluşturur

GET / POST / DELETE

Oku / Oluştur / Sil

Enum draft: 0

DB'de 0, kodda draft anlamına gelir

Strong Parameters

Sadece izin verilen alanlar geçer (güvenlik)

Partial _form

Dosya adı _ ile başlar, tekrar kullanılır

Bootstrap grid

12 sütun; col-md-6 = yarım genişlik

belongs_to

Foreign key bu tabloda bulunur

Validation vs Normalization

Validation reddeder, normalization düzenler

Authentication vs Authorization

Kimlik doğrulama vs yetki kontrolü

Internet Kaynaklari ile Derinlestirilmis Konular

PDF slayt basliklari (kapsulleme, migration, Strong Parameters vb.) resmi Ruby/Rails dokumantasyonu ve guvenilir kaynaklardan derlenerek aciklandi. Ornek kodlar sinava hazirlik icin guncellendi.

Tumu

Hafta 2

Hafta 3

Hafta 4

Hafta 5

Hafta 6

Hafta 7

Hafta 8

Hafta 9

Hafta 10

H2 Kapsulleme (Encapsulation)

H2 Kalitim / Miras (Inheritance)

H2 Polimorfizm

H2 MVC Mimarisi

H2 Symbol vs String

H3 ORM (Object Relational Mapping)

H3 Migration

H3 find / find_by / where

H4 Active Record Enum

H4 HTTP Metotlari ve CRUD

H4 ERB Sablonlari

H5 Strong Parameters

H5 form_with ve Partial

H6 Bootstrap Grid

H7 Model İlişkileri (belongs_to / has_many)

H7 HABTM (has_and_belongs_to_many)

H8 Validation (validates)

H8 Normalization (normalizes)

H8 Active Storage

H9 I18n (Yerelleştirme)

H9 FriendlyId (Slug URL)

H9 Action Text

H10 Devise (Authentication)

H10 Authorization (Yetkilendirme)

H2 Bloklar, Proc ve Lambda

SOLID İlkeleri — Odev / Sınav Formatı

SOLID İlkeleri

İnternet Programcılığı II — Nesne Yönelimli Tasarım

Ad Soyad: _____

Öğrenci No: _____

Tarih: _____

Aşağıda her SOLID ilkesi için: **ilke adı, tanımı, amacı, kötü/iyi Ruby kod örneği** ve açıklamaları yer alır. Yazdır butonu ile PDF olarak da alabilirsiniz.



Single Responsibility Principle (SRP)

Tek Sorumluluk İlkesi

İLKE TANIMI

Bir sınıf yalnızca tek bir sorumluluğa sahip olmalıdır; yani sınıfı değiştirmek için yalnızca tek bir nedeni olmalıdır.

AMACI / COZMEYE CALISTIĞI PROBLEM

Buyuyan sınıflarda kodun karışmasını, bakım zorluğunu ve hata riskini azaltmak. Her modul tek bir işi yapsın; değişiklikler izole kalsın.

Problem: Bir sınıf hem iş mantığı hem dosya kaydı hem e-posta gönderimi yaparsa her yeni gereksinimde tüm sınıf kırılabilir.

İLKEYE AYKIRI (KOTU) KOD

```
class Post
  def publish
    save_to_database
    send_email_to_subscribers
    write_to_log_file
  end

  def save_to_database
    # veritabanı işlemi
  end
end
```

```
def send_email_to_subscribers
  # e-posta gonderimi
end

def write_to_log_file
  # log yazma
end
end
```

Neden kotu? Post sinifi ayni anda veritabani, e-posta ve loglama sorumluluklarini tasiyor. E-posta servisi degisirse Post sinifi degismek zorunda kalir. Test etmek zorlasir; tek bir degisiklik birden fazla nedenden dolayi sinifi etkiler — SRP ihlali.

ILKEYE UYGUN (iyi) KOD

```
class Post
  def publish(publisher:, notifier:, logger:)
    publisher.save(self)
    notifier.notify_subscribers(self)
    logger.info("Post yayinlandi: \#{title}")
  end
end

class PostPublisher
  def save(post)
    # sadece veritabani
  end
end

class EmailNotifier
  def notify_subscribers(post)
    # sadece e-posta
  end
end

class FileLogger
  def info(message)
    # sadece log
  end
end
```

Neden iyi? Her sinifin tek gorevi var: Post yayinlama kararini verir, diger isleri uzman siniflara devreder. E-posta mantigi degistiginde yalnızca EmailNotifier guncellenir. Siniflar kucuk, test edilebilir ve bakimi kolay — SRP'ye uygun.



Open/Closed Principle (OCP)

Acik/Kapali Ilke

Yazılım varlıkları (sınıflar, modüller) genişlemeye açık, değişikliğe kapalı olmalıdır. Mevcut kodu değiştirmeden yeni davranış eklenebilmelidir.

AMACI / COZMEYE CALISTIĞI PROBLEM

Calisan kodu bozmadan yeni ozellik eklemek. if/elsif zincirleri yerine genisleme (kalitim, modul, polymorphism) ile yeni davranislar eklenir.

Problem: Her yeni odeme tipi icin mevcut sinifa if/elsif eklenirse kod surekli degisir ve regression riski artar.

ILKEYE AYKIRI (KOTU) KOD

```
class PaymentProcessor
  def pay(method, amount)
    if method == :credit_card
      # kredi karti odeme
    elsif method == :paypal
      # paypal odeme
    elsif method == :bank_transfer
      # havale – her yeni tip icin buraya ekleme yapilir
    end
  end
end
```

Neden kotu? Yeni bir odeme yontemi (ornegin :crypto) eklendiginde PaymentProcessor sinifinin icindeki pay metodu degistirilmek zorunda. Acik/kapali ilkeye gore sinif genislemeye kapali degil, her ekleme mevcut kodu kirar — OCP ihlali.

ILKEYE UYGUN (iyi) KOD

```
class PaymentProcessor
  def pay(gateway, amount)
    gateway.charge(amount)
  end
end

class CreditCardGateway
  def charge(amount)
    # kredi karti
  end
end

class PaypalGateway
  def charge(amount)
    # paypal
  end
end

# Yeni tip: mevcut kodu degistirmeden ekle
class CryptoGateway
  def charge(amount)
    # kripto odeme
  end
end
```

```
end
end
```

Neden iyi? PaymentProcessor artık hangi ödeme tipi olduğunu bilmez; charge metoduna sahip her gateway çalışır. CryptoGateway eklemek için PaymentProcessor'a dokunulmaz. Mevcut kod kapalı (değişmez), yeni davranış genişlemeyle açık — OCP'ye uygun.



Liskov Substitution Principle (LSP)

Liskov Yerine Koyma İlkesi

İLKE TANIMI

Alt sınıf nesneleri, üst sınıf nesnelerinin yerine kullanılabilir; programın davranışı bozulmamalı.

AMACI / COZMEYE CALISTIĞI PROBLEM

Kalıtım hiyerarsisinde alt sınıfın üst sınıfın sözleşmesini (contract) bozmasını sağlamak. Polimorfizm güvenli çalışsın.

Problem: Alt sınıf üst sınıfın beklediği davranışı değiştirirse (örneğin fly metodu hata fırlatırsa) polimorfizm kırılır.

İLKEYE AYKIRI (KOTU) KOD

```
class Bird
  def fly
    "Ucuyor"
  end
end

class Penguin < Bird
  def fly
    raise "Penguin ucamaz!" # üst sınıfın sözleşmesini bozar
  end
end

birds = [Bird.new, Penguin.new]
birds.each { |b| b.fly } # Penguin'de patlar
```

Neden kötü? Penguin, Bird'in yerine konulduğunda fly çağrısı programı kırar. Üst sınıf 'her kuş uçabilir' beklentisi vardır; alt sınıf bunu ihlal eder. LSP: alt sınıf üst sınıfın yerine güvenli kullanılamıyor.

İLKEYE UYGUN (İYİ) KOD

```
class Bird
  def move
    raise NotImplementedError
  end
end
```

```

class Sparrow < Bird
  def move
    "Ucuyor"
  end
end

class Penguin < Bird
  def move
    "Yuzuyor"
  end
end

birds = [Sparrow.new, Penguin.new]
birds.each { |b| puts b.move } # ikisi de calisir

```

Neden iyi? Ortak sozlesme move metodudur; her alt sinif kendi gercegini uygular. Penguin Bird yerine konuldugunda program bozulmaz. Alt sinif ust sinifin bekledigi arayuzu korur — LSP'ye uygun.



Interface Segregation Principle (ISP)

Arayuz Ayirimi Ilkesi

ILKE TANIMI

Istemiciler kullanmadiklari arayuzlere bagimli olmamali. Buyuk, siskin arayuzler yerine kucuk, ozellestirilmis arayuzler tercih edilmeli.

AMACI / COZMEYE CALISTIGI PROBLEM

Siniflari zorunlu ama bos/kullanilmayan metot implementasyonlarından kurtarmak. Modul ve mixin'lerde sadece gereken davranislar sunulur.

Problem: Tek dev modul hem yazdir hem taray hem faks metotlari icerirse bazı siniflar bos metot birakmak zorunda kalir.

ILKEYE AYKIRI (KOTU) KOD

```

module Worker
  def work; end
  def eat; end
  def sleep; end
  def attend_meeting; end # her worker icin gerekli degil
end

class Robot
  include Worker

  def work
    "Calisiyor"
  end

  def eat

```

```
# robot yemek yemez – bos veya anlamsiz
end

def sleep
  # robot uyumaz
end

def attend_meeting
  # anlamsiz
end
end
```

Neden kotu? Robot, kullanmadigi eat/sleep/attend_meeting metotlarini da implement etmek zorunda. Buyuk arayuz istemciyi gereksiz bagimliliklara zorlar. ISP: kullanilmayan metotlar arayuze dahil edilmemeli.

ILKEYE UYGUN (iyi) KOD

```
module Workable
  def work; end
end

module Eatable
  def eat; end
end

class Human
  include Workable
  include Eatable

  def work; "Calisiyor"; end
  def eat; "Yemek yiyor"; end
end

class Robot
  include Workable

  def work; "Calisiyor"; end
  # sadece ihtiyaci olan modul
end
```

Neden iyi? Arayuzler role gore ayrildi: Workable, Eatable. Robot yalnızca Workable alır; bos eat/sleep yazmak zorunda kalmaz. Her sinif sadece ihtiyaci olan davranisa bagimli — ISP'ye uygun.



Dependency Inversion Principle (DIP)

Bagimlilik Tersine Cevirme Ilkesi

ILKE TANIMI

Ust seviye moduller alt seviye modullere bagimli olmamali; her ikisi de soyutlamalara (abstraction) bagimli olmalidir.

AMACI / COZMEYE CALISTIGI PROBLEM

Siki bagimliliklari (concrete class) gevsetmek; test, degistirme ve genisletmeyi kolaylastirmak. Dependency Injection ile somut sinif yerine arayuz/soyut bagimlilik kullanilir.

Problem: Controller dogrudan PostgreSQL sinifina bagliysa MySQL'e gecis tum controller'i degistirir.

ILKEYE AYKIRI (KOTU) KOD

```
class PostsController
  def create
    db = PostgreSQLConnection.new # somut sinifa dogrudan bagimli
    db.insert(params[:post])
  end
end
```

Neden kotu? PostsController dogrudan PostgreSQLConnection somut sinifina bagimli. Veritabani degisirse controller degismek zorunda. Ust seviye (controller) alt seviyeye (PostgreSQL) bagimli — DIP ihlali.

ILKEYE UYGUN (iyi) KOD

```
class PostsController
  def initialize(repository:)
    @repository = repository # soyut bagimlilik enjekte edilir
  end

  def create(params)
    @repository.insert(params)
  end
end

class PostRepository
  def insert(data)
    # PostgreSQL veya baska implementasyon
  end
end

# Rails'de benzeri:
# @post = Post.new(post_params) → ActiveRecord soyutlamasi
PostsController.new(repository: PostRepository.new)
```

Neden iyi? Controller somut veritabani sinifini bilmez; repository arayuzune bagimlidir. Testte sahte (mock) repository enjekte edilebilir. Ust ve alt seviye soyutlamaya bagimli — DIP'ye uygun. Rails'de ActiveRecord da bu soyutlamadir.

Hafta 2 — Ruby & Rails Giriş

Bu haftada ne öğreniyorsun? Ruby dilini öğrenir, Rails projesini kurar ve MVC yapısını anlarsın.

Her şey nesne

MVC akisi

rails s

Kısa Video Özeti (~1 dk) — Hafta 2



0:00 / 1:12



Bu hafta — internet kaynaklı açıklamalar:

Kapsulleme (Encapsulation)

Kalitim / Miras (Inheritance)

Polimorfizm

MVC Mimarisi

Symbol vs String

Bloklar, Proc ve Lambda

Basit Anlatım

- Ruby'de sayı, metin, dizi — hepsi nesnedir.
- Rails = Model (veri) + View (arayüz) + Controller (mantık).
- routes.rb istegi yönlendirir, controller işler, view gösterir.
- rails new, db:create, rails s ile proje çalıştırılır.

Rails blog uygulaması, postgresql veritabanı ve bootstrap kütüphanesi

–

kullanılarak aşağıdaki gibi oluşturulur. blogger-a-ogrencino Github'da oluşturulan repo ismiyle aynı olduğuna dikkat edin. Dikkat! ogrencino kısmı kendi öğrenci numaranız olacak.

Yukarıdaki kodu çalıştırdığınızda blogger-a-ogrencino adında bir rails projesini ilklendirmiş (başlatmış) olacaksınız.

Ruby

+

Ruby Veri Türleri – Array

+

Ruby Veri Türleri – Array

+

Ruby Veri Türleri – Array

+

Ruby Veri Türleri – Array

+

Ruby Veri Türleri – Float

+

Ruby Veri Türleri – Hash

+

Ruby Veri Türleri – Integer

+

Ruby Veri Türleri – Integer

+

Ruby Veri Türleri – String

+

Ruby Veri Türleri – String

+

Ruby Veri Türleri – String

+

Ruby Veri Türleri – Symbol

+

Rubygems

+

RubyGems, kütüphanelerin oluşturulması, paylaşılması ve kurulumu için tasarlanmış

+

Erişim Metotları Public, Private ve Protected

+

Metot Yazma Kuralları

+

Değişken Sayıda Argümanlar

+

Global Değişkenler ve Sabitler (Constants)

+

Local Değişkenler

+

Sınıf Değişkeni

+

Github Classroom Proje Depo Oluşturma Linki

+

Rails Veritabanı Kullanıcı Adı ve Parolayı Credentials’a Kaydetme

+

Sınıflar (Classes)

+

Son Kontroller

+

Modül (Module)

+

Modül (Module) Namespacing

+

Bloklar

+

Bloklar

+

Bloklar

+

Bloklar do/end ile ya da { } ile sarmalanmışlardır. Bloklar için each, map,

+

Veritabanı username ve password belirttikten sonra artık veritabanları

+

veritabanı username ve password bilgileri eklenmesi gerekmektedir.

+

Koşullar

+

Rails blog uygulamasında yapacağımız bütün geliştirmeler için artık bu

+

Rails Blog Uygulamasının Oluşturulması

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı

+

Rails Dizin Yapısı – config

+

Rails Dizin Yapısı – config

+

Rails Dizin Yapısı – config

+

Rails Anasayfa Güncelleme

+

Rails MVC

+

Veritabanı şemasını nasıl değiştirileceğini tanımlayan

+

2. Hafta Ders İçeriği

+

Blog Uygulamasının Github'a Gönderilmesi

+

Blog Uygulamasının Github'a Gönderilmesi

+

Blog Uygulamasının Veritabanı Ayarı

+

Fonksiyonlar (Methods)

+

for Döngüsü

+

Kaynaklar

+

Proc (Procedures) ve Lambda

+

Proc (Procedures) ve Lambda Farkları

+

Rails uygulamasının

+

Rails Uygulamasının Çalıştırılması

+

ternary Operatörü

+

Veritabanı için ön tanımlı verilerin oluşturulduğu dosya

+

Veritabanı şeması

+

Veritabanı şifreleri, API anahtarları vs. gizli

+

Hafta 3 — ORM & Active Record

Bu haftada ne ogreniyorsun? SQL yazmadan veritabani islemleri yaparsin. Tablo = Model sinifi.

ORM

find vs where

migration

Kisa Video Ozeti (~1 dk) — Hafta 3

▶ 0:00 / 0:53



Bu hafta — internet kaynakli aciklamalar:

ORM (Object Relational Mapping)

Migration

find / find_by / where

Basit Anlatim

- ORM: veritabani tablosu = Ruby sinifi.
- Post.create ile kayıt ekle, Post.all ile listele.
- find id arar (hata verir), find_by arar (nil doner), where filtreler.
- Migration ile tablo/kolon ekle; rails db:migrate calistir.

Active Record, kalıcı depolama gerektiren Ruby nesnelerini bir veritabanına

–

kaydetmenize ve kullanmanıza yardımcı olur.

Business Logic nedir?

Verilerin nasıl oluşturulacağını, depolanacağını ve değiştirileceğini belirleyen iş kurallarını kodlayan programın bir parçasıdır.

ORM (Object Relational Mapping) Nedir?

+

ORM (Object Relational Mapping) Nedir?

+

ORM (Object Relational Mapping) Nedir?

+

ORM (Object Relational Mapping) Nedir?

+

ORM ile SQL sorgusu yazmadan veritabanına DDL (Data Definition Language)

+

ORM Kullanmanın Avantajları

+

Rails Migration Status

+

Rails Migration Status

+

Active Record'un Desteklediği Veri Türleri

+

Active Record'un Desteklediği Veri Türleri

+

Active Record'un Desteklediği Veri Türleri

+

Active Record'un Desteklediği Veri Türleri

+

Migration dosyası ile yapılan değişiklikleri geri almak için rollback yapılır.

+

Migration sınıfları ActiveRecord::Migration sınıfını miras alır ve change adında

+

Rails Model Oluşturma

+

Veritabanındaki her bir tablo bir sınıfa karşılık gelmektedir.

+

Veri Tabanı Tablosu Oluşturma

+

Veri Tabanı Tablosu Oluşturma

+

Veri tabanına bağlanıp tabloları görüntüleyin:

+

Veritabanındaki posts tablosunun kolonlarını görüntüleyelim.

+

Konsolda Rails Modeliyle Etkileşim

+

Konsolda Rails Modeliyle Etkileşim

+

Konsolda Rails Modeliyle Etkileşim

+

Konsolda Rails Modeliyle Etkileşim

+

Veritabanından bir kayıt silmek için destroy kullanılır.

+

Where and Kullanımı

+

Where not ve Where or Kullanımı

+

Kayıtları Güncelleme

+

Kayıtları Güncelleme

+

Kayıtları güncellemenin 2 yolu var. Biri update kullanmak bir diğeri nesneye

+

Kayıtları Silmek

+

Migration bir veritabanı şemasının (schema) nasıl değiştirileceğini tanımlayan

+

Veritabanından Verilere Ulaşılması

+

Rails Migration Yapısı

+

Rails Migration Yapısı

+

Rails Migration Yapısı

+

Rails Migration Yapısı

+

Rails Migration Yapısı

+

Rails Migration Yapısı

+

3. Hafta Ders İçeriği

+

ActiveRecord Nedir?

+

ActiveRecord Nedir?

+

ActiveRecord::Relation

+

Genel

+

Post modelini oluşturduktan sonra uygun bir mesaj ile yapılanları Github'a

+

Spesifik Bir Kayıt Bulma

+

Spesifik Bir Kayıt Bulma

+

Spesifik Bir Kayıt Bulma

+

Tür Açıklama

+

Tür Açıklama

+

Tür Açıklama

+

Tür Açıklama

+

Veri tabanına kaydedildikten sonra post içeriği:

+

Veri tabanına kaydetmek için save etmeliyiz.

+

Veritabanı Üzerinde Yapılabilecek İşlemler

+

Veritabanı Üzerinde Yapılabilecek İşlemler

+

Veritabanındaki Kayıtlara Model Üzerinden Ulaşmak

+

Veritabanındaki Kayıtlara Model Üzerinden Ulaşmak

+

Veritabanındaki Kayıtlara Model Üzerinden Ulaşmak

+

Veritabanındaki Verileri Filtreleme

+

Veritabanındaki Verileri İstenen Sırada Getirme

+

Hafta 4 — Enum, Routes, Controllers

Bu haftada ne ogreniyorsun? URL'leri controller'a baglarsin. 7 CRUD islemi resources ile gelir.

[HTTP metotlari](#)[resources :posts](#)[Enum](#)

Kisa Video Ozeti (~1 dk) — Hafta 4



0:00 / 0:49



Bu hafta — internet kaynakli aciklamalar:

[Active Record Enum](#)[HTTP Metotlari ve CRUD](#)[ERB Sablonlari](#)

Basit Anlatim

- GET okur, POST olusturur, PUT/PATCH gunceller, DELETE siler.
- resources :posts otomatik 7 route olusturur.
- Enum: sayisal degerlere anlamlı isim verir (draft: 0).
- @degisken controller'dan view'a veri tasir.

ERB (Embedded Ruby) Nedir?

İçeriğimizin gösterileceği html dosyası views/posts/index.html.erb erb uzantılıdır.

ERB, ruby kodunu çalıştırarak Rails ile dinamik olarak HTML üretmemize olanak tanır.

RUBY

```
<%= %> etiketi, ERB'ye içerideki Ruby kodunu çalıştırmasını ve dönüş değerini çıktı olarak vermesini söyler.  
<h1>Posts</h1>  
<% @posts.each do |post| %>  
<%= post.title %><br/>  
<%= post.article %><br/>  
<%= post.status %><br/>  
<% end %>  
views/posts/index.html.erb
```

İNTERNET KAYNAĞI İLE ACIKLAMA

ERB Sablonlari — <% %> Ruby kodu çalıştırır, <%= %> çıktıyı HTML'e yazar.

ERB (Embedded Ruby) view dosyalarında Ruby kodu çalıştırır. <% if @post %> sadece çalıştırır, <%= @post.title %> HTML'e escape edilmiş çıktı yazar. <%- -%> boşluk kırpmaya için kullanılabilir.

ORNEK KOD

```
<%# yorum %>  
<h1><%= @post.title %></h1>  
<% @posts.each do |post| %>  
  <p><%= link_to post.title, post %></p>  
<% end %>
```

[Rails Action View Overview](#)

Index sayfasında sadece status durumu published postlar gösterilmesi için:

Post modeline status alanı ekleme

+

Post modeline status ekledikten sonra kolon bilgileri aşağıdaki gibi

+

Active Record Enum

+

Active Record Enum – Prefix

+

Active Record Enum – Suffix

+

Active Record Scope

+

Active Record Scope

+

Active Record Scope

+

Active Record Scopes

+

HTTP Metot ve Amaçları

+

HTTP Metot ve Amaçları

+

HTTP Metotlar ve Yollar (Routes)

+

Link Helpers

+

HTTP (Hyper Text Transfer Protocol)

+

http Protokol (Hyper-Text Transfer Protocol)

+

Rails Router

+

Rails Routes

+

Rails Routes

+

Rails Routes

+

Rails Routes

+

Rails Routes

+

Rails Routes – Resources

+

Veritabanına Seed Verisi Ekleme

+

Veritabanına Seed Verisi Ekleme

+

4. Hafta Ders İçeriği

+

Arayüzden Blog Girdisi Oluşturma

+

Belirli Bir Kaydın Görüntülenmesi

+

Belirli Bir Kaydın Görüntülenmesi

+

Index – Bütün post kayıtlarını gösterir.

+

Index Sayfa İçeriği

+

Post Modeli Sütun Bilgileri

+

URL Parçaları

+

Veritabanındaki bütün postları listeler.

+

Veritabanındaki verileri html sayfasında gösterelim.

+

Hafta 5 — Form Helpers & Strong Parameters

Bu haftada ne ogreniyorsun? Form ile veri alir, Strong Parameters ile guvenli kaydedersin.

[form_with](#)[Strong Params](#)[partial](#)

Kisa Video Ozeti (~1 dk) — Hafta 5

▶ 0:00 / 0:47



Bu hafta — internet kaynakli aciklamalar:

[Strong Parameters](#)[form_with ve Partials](#)

Basit Anlatim

- `form_with` model: `@post` ile form olusturulur.
- `post_params` sadece izin verilen alanlari gecirir.
- `Partial` (`_form`) ile tekrar eden kod azaltilir.
- `before_action` ile ortak kod tek yerde toplanir.

Action View

–

- MVC’de View’e karşılık gelir.
- Web isteklerini karşılamak için Action Controller ile birlikte çalışırlar.
- Action Controller, model katmanı ile iletişim kurmak ve veri almakla ilgilenir.
- Action View bu verileri kullanarak web isteğine bir yanıt gövdesi oluşturmaktan sorumludur.
- Action View, formlar, tarihler ve stringler için HTML etiketlerini dinamik olarak oluşturan bir çok yardımcı metot (helper) sağlar.

Action View Form Helpers

+

Action View Form Helpers

+

Strong Parameter Nedir?

+

Index Sayfasına Yeni Post Ekleme Linki Ekle

+

Partials

+

Partials

+

Before Action Callback

+

Before Action Callback

+

button_to Kullanımı

+

Arayüzden Post Güncellemek

+

MVC İçeriklerinin Son Hali

+

Arayüzden Post Kaydını Silmek

+

5. Hafta Ders İçeriği

+

Arayüzden Post Oluşturmak

+

Kaynaklar

+

Hafta 6 — Bootstrap Frontend

Bu haftada ne ogreniyorsun? Blog arayuzunu Bootstrap ile duzenlersin.

[12 sutun grid](#)[navbar/footer](#)[card](#)

Kisa Video Ozeti (~1 dk) — Hafta 6



0:00 / 0:36



Bu hafta — internet kaynakli aciklamalar:

[Bootstrap Grid](#)

Basit Anlatim

- Grid 12 sutun: col-md-6 = yarim genislik.
- container ortalar, container-fluid tam genislik.
- Navbar ve footer partial olarak layout'a eklenir.
- card, btn siniflari ile modern gorunum.

Bootstrap

Hızlı ve kolay bir şekilde web uygulaması geliştirebilmek için kullanılan ücretsiz bir frontend çerçevesidir. Formlar, butonlar, tablolar, navigation, modals gibi html ve css templateleri mevcuttur ve isteğe bağlı bazı JS eklentileri de mevcuttur.

INTERNET KAYNAĞI İLE ACIKLAMA

Bootstrap Grid — 12 sütunluk responsive grid; container, row, col-* sınıfları.

Bootstrap 5 grid sistemi 12 sütun üzerinden çalışır. container içeriği ortalar, row satır, col-md-6 medium ekranda yarım genişlik demektir. Rails 7+ ile rails new --css bootstrap veya importmap ile eklenir.

ORNEK KOD

```
<div class="container">
  <div class="row">
    <div class="col-md-8">Ana icerik</div>
    <div class="col-md-4">Sidebar</div>
  </div>
</div>
```

[Bootstrap Grid Docs](#)

Bootstrap 5 mobil cihazlara duyarlı olacak şekilde tasarlanmıştır.

+

Bootstrap Breakpoints (Kırılma Noktaları)

+

Bootstrap Container

+

Bootstrap Grid

+

Bootstrap Grid

+

Bootstrap Grid

+

Bootstrap ile Footer Ekleyelim

+

Bootstrap ile Navbar Ekleyelim

+

Bootstrap İnceleyin

+

Bootstrap JS

+

Bootstrap JS işlemleri düzgün çalışması için popper.js kütüphanesi

+

Form Sayfasını Güncelleyelim

+

Footer veya Navbar'a linkleyin. Linke basınca ilgili sayfa açılsın.

+

Hakkımda sayfası için path, view ve controller'da ilgili action'ı oluşturun.

+

Navbar'da görünmesini istediğiniz başka alanlar varsa link olarak

+

Projelerim sayfası için path, view ve controller'da ilgili action'ı oluşturun.

+

Footer için views/shared klasörü altında _footer.html.erb adında partial

+

Navbar'daki Blogger yazısına (başka bir isim de verebilirsiniz) basınca

+

image_tag HTML resim etiketi oluşturur. Resimler app/assets/images altına

+

image_tag Kullanımı

+

image_tag metodu ile kullanın.

+

Index Sayfasını Güncelleyelim

+

Index Sayfasını Güncelleyelim

+

Show Sayfasını Güncelleyelim

+

6. Hafta Ders İçeriği

+

Hakkımda

+

Hakkımda

+

Hakkımda Sayfası

+

Hakkımda Sayfası

+

Not

+

Projelerim Sayfası

+

Projelerim Sayfası

+

Hafta 7 — Model İlişkileri & Kategori

Bu haftada ne öğreniyorsun? Tablolar arası bağlantı kurarsın. Post-Category çoktan coga.

belongs_to/has_many

HABTM

scaffold

Kisa Video Özeti (~1 dk) — Hafta 7



0:00 / 0:34



Bu hafta — internet kaynaklı açıklamalar:

Model İlişkileri (belongs_to / has_many)

HABTM (has_and_belongs_to_many)

Basit Anlatım

- belongs_to: FK bu tabloda. has_many: karşı taraf çok kayıt.
- HABTM: ara tablo, id yok, iki FK var.
- scaffold ile model+view+controller hızlı oluşur.
- category_ids: [] ile çoklu kategori seçilir.

Form Sayfasına Kategori Select Box Ekleme

–

Rails Model ilişkileri – belongs_to

+

Rails Model ilişkileri – has_and_belongs_to_many

+

Rails Model ilişkileri – has_and_belongs_to_many

+

Rails Model ilişkileri – has_many ve belongs_to

+

Rails Model ilişkileri – has_one ve belongs_to

+

kategorileri seçilebildi mi? (Strong Parameter 🌞

+

Rails Model ilişkileri – has_many

+

Rails Model ilişkileri – has_many :through

+

Rails Model ilişkileri – has_many :through

+

Controller Güncelleme

+

Footer'a Kategoriler linki ekleyin.

+

Navbar'a İşlemler için dropdown menü ekleyelim. Yeni post ve kategori ekleme linkleri

+

Rails Model ilişkileri – has_one

+

Rails Model ilişkileri – has_one :through

+

Rails Model ilişkileri – has_one :through

+

Rails Model ilişkileri – has_one :through

+

Tablolar Arası Bağlantı Sağlayalım

+

Tablolar Arası Bağlantı Sağlayalım

+

Tablolar Arası Bağlantı Sağlayalım

+

Kategori görüntüleme sayfası örnek url: localhost:3000/categories/1

+

Kategori Sayfaları

+

Kategoriler Sayfasında İlgili Postları Listelemek

+

Post Girdisinin Kategorisini Belirleme

+

Post Show Sayfasına Kategoriler İçin Badge Ekleme

+

ActiveRecord::Migration[8.0]

+

7. Hafta Ders İçeriği

+

Rails Model ilişkileri

+

Hafta 8 — Validation & Active Storage

Bu haftada ne ogreniyorsun? Veri kurallarini modelde tanimlarsin. Resim yuklemek icin Active Storage.

validates

normalizes

Active Storage

Kisa Video Ozeti (~1 dk) — Hafta 8

▶ 0:00 / 0:33



Bu hafta — internet kaynakli aciklamalar:

Validation (validates)

Normalization (normalizes)

Active Storage

Basit Anlatim

- validates: veriyi kontrol eder, gecersizse kaydetmez.
- normalizes: kaydetmeden once duzenler (bosluk sil, buyuk harf).
- presence, uniqueness, length en sik kullanilan kurallar.
- has_one_attached :image ile dosya yukleme.

Active Record, her validation helper için varsayılan hata mesajını

–

kullanır. Ancak :message seçeneği ile doğrulama başarısız olduğunda hatalar koleksiyonuna eklenecek mesajı kendimiz belirleyebiliriz.

RUBY

```
class Post < ApplicationRecord
  validates :title, presence: { message: "Post başlığı boş bırakılamaz."}
end
```

INTERNET KAYNAGI İLE ACIKLAMA

Validation (validates) — Model kaydetmeden önce veri kurallarını kontrol eder.

validates :title, presence: true boş bırakılamaz der. uniqueness tekil olmayı, length uzunluk sınırını kontrol eder. Gecersiz kayıt save → false döner, errors mesajları dolar.

ORNEK KOD

```
class Post < ApplicationRecord
  validates :title, presence: true, length: { minimum: 3 }
  validates :slug, uniqueness: true
end

post = Post.new
post.valid?  #=> false
post.errors.full_messages
```

[Rails Validations Guide](#)

Kategori modelindeki name alanının boş geçilmemesini ve normalizasyon

+

Kategori Validation ve Normalizasyon

+

Post Modeli Validation ve Normalizasyon Son Hali

+

Post modelinin title alanı için oluşturulan normalizasyon ile title bilgi

+

Rails Normalization

+

Rails Normalization

+

Rails Validation

+

Rails Validation

+

Validation Helpers

+

Validation Helpers – exclusion

+

Validation Helpers – format

+

Validation Helpers – inclusion

+

Validation Helpers – length

+

Validation Helpers – length

+

Validation Helpers – numericality

+

Validation Helpers – presence

+

Validation Helpers – uniqueness

+

Validation Helpers – uniqueness

+

Validation Options

+

Validation Options – allow_blank

+

Validation Options – allow_nil

+

Validation Options – if ve unless

+

Validation Options – message

+

Validation Options – on

+

Active Record validasyonları veritabanına eklenecek verilerin belirli kurallara

+

Validasyonları Atlayan Metotlar

+

Hata Mesajları

+

Hata Mesajları

+

Hata Mesajları

+

Sayfalarda Hata Mesajlarının Gösterimi

+

Sayfalarda Hata Mesajlarının Gösterimi

+

Sayfalarda Hata Mesajlarının Gösterimi

+

Active Storage

+

Active Storage – Dosya Attach Etme

+

Active Storage – Dosya Attach Etme

+

Active Storage, dosyaları Amazon S3, Google Cloud Storage veya Microsoft

+

8. Hafta Ders İçeriği

+

Post modelindeki title alanı hem benzersiz, hem boş geçilemez hem de

+

Tetikleyici (Trigger) Validasyonlar

+

veritabanına eklenir.

+

veritabanına kaydedilebilir.

+

veritabanına kaydedilmeden önce string'in başında, sonunda veya ortasında

+

Hafta 9 — I18n, FriendlyId, Action Text

Bu haftada ne öğreniyorsun? Çoklu dil, SEO URL ve zengin metin editörü.

[I18n.t](#)[FriendlyId](#)[Action Text](#)

Kisa Video Özeti (~1 dk) — Hafta 9



0:00 / 0:35



Bu hafta — internet kaynaklı açıklamalar:

[I18n \(Yerelleştirme\)](#)[FriendlyId \(Slug URL\)](#)[Action Text](#)

Basit Anlatım

- I18n.t ile metin çevirirsin (tr.yml, en.yml).
- ?locale=en ile dil değiştirilir.
- FriendlyId: /posts/1 yerine /posts/baslik-slug.
- Action Text: zengin metin editörü (kalın, liste, resim).

Uygulamada diller arası geçiş nasıl yapılacak?

```
around_action :switch_locale
```

```
private
```

RUBY

```
def switch_locale(&action)
  locale = params[:locale] || I18n.default_locale
  I18n.with_locale(locale, &action)
end

application_controller.rb
application_controller dosyasına yukarıdaki kodları ekleyelim.
```

Y erelleştirme Dosya Yapısı

config

locales

| -defaults

| ---tr.yml

| ---en.yml

| -models

| ---post

| -----tr.yml

| -----en.yml

| -views

| ---defaults

| -----tr.yml

| -----en.yml

| ---posts

| -----tr.yml

| -----en.yml

Y erelleştirmede Default Değerler

config/locales/defaults/tr.yml dosyası oluşturalım ve

<https://github.com/svenfuchs/rails-i18n/blob/master/rails/locale/tr.yml>

adresindeki içeriği kopyalayıp dosya içerisine kaydedelim.

config/locales/defaults/en.yml dosyası oluşturalım ve

<https://github.com/svenfuchs/rails-i18n/blob/master/rails/locale/en.yml>

adresindeki içeriği kopyalayıp dosya içerisine kaydedelim.

config/locales/defaults/tr.yml dosyasının en üstüne eylemler (actions)

için şunları ekleyin:

tr:

actions:

name: İşlemler

back: Geri

destroy: Sil

edit: Düzenle

new: Ekle

reset: Sıfırla

search: Ara

show: Görüntüle

update: Güncelle

Y erelleştirmede Default Değerler

config/locales/defaults/en.yml dosyasının içine eylemler (actions) için

şunları ekleyin:

en:

actions:

name: Actions

back: Back

destroy: Destroy

edit: Edit

new: New

reset: Reset

search: Search

show: Show

update: Update

Y erelleştirmede Default Değerler

Y erelleştirmelerin Sayfalarda Kullanımı

Bütün görüntüleme linklerini güncelleyelim. Örnek Post index sayfasındaki

show linki.

```
<%= link_to t("actions.show"), post_path(post), class:"btn btn-info
```

```
btn-sm" %>
```

INTERNET KAYNAGI İLE ACIKLAMA

I18n (Yerelleştirme) — Çoklu dil desteği; I18n.t ve config/locales/*.yml dosyaları.

I18n modülü metinleri YAML dosyalarında tutar. I18n.t('posts.title') ceviri doner. URL'de ?locale=en veya session ile dil degistirilir. Rails varsayılan locale config/application.rb'de ayarlanır.

ORNEK KOD

```
# config/locales/tr.yml
tr:
  posts:
    title: "Basliklar"

# view
<%= t('posts.title') %>
<%= link_to 'EN', url_for(locale: :en) %>
```

[Rails I18n Guide](#)

Internationalization (I18n) – Y erelleştirme

+

Internationalization (I18n) – Y erelleştirme

+

Yerelleştirme ayarlarının yapılması gerekmektedir. locale.rb dosyasını

+

Footer için eklenen yerelleştirmeleri kullanalım.

+

Footer’da gösterilen linkleri yerelleştirelim.

+

yerelleştirmelerini yapın.

+

Controller Düzenleme

+

Post show sayfasındaki edit, back ve destroy linklerini güncelleyelim.

+

footer:

+

footer:

+

Footer'a Türkiye ve ABD bayrak iconu kullanarak bunlara

+

Kategori sayfalarının post modelinde olduğu gibi bütün

+

Kategori Sayfalarının Y erelleştirilmesi

+

Kategori View Y erelleştirme

+

Kategori View Y erelleştirme

+

9. Hafta Ders İçeriği

+

activerecord:

+

activerecord:

+

activerecord:

+

activerecord:

+

Blog Girdilerinde Zengin İçerik Üretimi

+

Blog Girdilerinde Zengin İçerik Üretimi

+

Blog Girdilerinde Zengin İçerik Üretimi

+

Category Model Y erelleřtirme

+

Category Model Y erelleřtirme

+

Modele Ekleyelim

+

Hafta 10 — Devise & Authorization

Bu haftada ne ogreniyorsun? Kullanici girisi ve rol bazli yetkilendirme.

Devise

authenticate_user!

roller

Kisa Video Ozeti (~1 dk) — Hafta 10

▶ 0:00 / 0:41



Bu hafta — internet kaynakli aciklamalar:

Devise (Authentication)

Authorization (Yetkilendirme)

Basit Anlatim

- Devise: kayıt ol, giriş yap, şifre sıfırla.
- Authentication = kimlik, Authorization = yetki.
- authenticate_user! giriş zorunlu kılar.
- current_user.posts.new ile post kullanıcıya atanır.

devise :database_authenticatable, :registerable,

–

:recoverable, :rememberable, :validatable,

:timeoutable

RUBY

end

app/models/user.rb

User modeline gender (cinsiyet) için parametreleri enum olarak ekleyelim ve

INTERNET KAYNAGI İLE ACIKLAMA

Devise (Authentication) — Hazır kullanıcı girişi: kayıt, giriş, şifre sıfırlama.

Devise gem'i User modeline moduller ekler (:database_authenticatable, :registerable vb.).

authenticate_user! girişi zorunlu kılar. current_user oturum açmış kullanıcıyı döner. devise_for

:users routes.rb'ye login/register yollarını ekler.

ORNEK KOD

```
class ApplicationController < ActionController::Base
  before_action :authenticate_user!
end

class PostsController < ApplicationController
  def create
    @post = current_user.posts.new(post_params)
  end
end
```

[Devise GitHub Wiki](#)

[Devise Getting Started](#)

devise :database_authenticatable, :registerable,

+

Devise Gem Ekle

+

Devise Gem Ekle

+

Devise gem'ini Gemfile'e ekleyelim: <https://github.com/heartcombo/devise>

+

Devise için default çeviriler:

+

Devise Kütüphanesi

+

Devise Sayfalarını Düzenle

+

Devise Sayfalarını Düzenle

+

Devise Strong Parameter

+

Devise view sayfalarını oluşturun.

+

Devise, Rails uygulamaları için tam özellikli bir kimlik doğrulama (Authentication) çözümüdür.

+

`devise_parameter_sanitizer.permit(:sign_in, keys: [:email,`

+

`devise_parameter_sanitizer.permit(:sign_up, keys: [:firstname,`

+

Authentication

+

Authentication ve Authorization Farkları

+

Post için Strong Parameter

+

Navbar'ın güncel halini aşağıdaki linkteki koddan yardım alarak

+

Post Controller Düzenleme

+

Post modeline eklenen user_id bilgisini controllers/posts_controller.rb içindeki

+

Rol Ekleme

+

URL düzenlemesi için friendly_id kütüphanesini kullanalım.

+

Admin Altında Kullanıcı Bilgileri

+

Admin Altında Kullanıcı Bilgileri

+

Admin Altında Kullanıcı Bilgileri

+

Admin Altında Kullanıcı Bilgileri

+

Admin Altında Kullanıcı Sayfaları

+

Navbar Güncelle

+

Navbar Güncelle

+

navbar'da gözükecek.

+

Post ile User Arasında İlişkileri Ekleyelim

+

Migration dosya içerisinde eklediğimiz bazı alanların düzenlenmesi:

+

activerecord:

+

activerecord:

+

Genel

+

kaynaklara veya işlemlere erişim hakkı

+

Kullanıcı Kaydı Gerçekleştirin

+

Post Modeline Kullanıcı Bilgisi Ekleme

+

Kendini Test Et — 39 Soru

Soruya tikla, cevabi gor. Sinav oncesi tum sorulari en az bir kez coz.

1 Ruby'de 3.class.superclass.superclass ne doner?

Object doner. Hiyerarsi: Integer → Numeric → Object → BasicObject. Yani 3 bir Integer nesnesidir ve ust sinifi Numeric, onun ust sinifi Object'tir.

2 Symbol ile String arasindaki temel fark nedir?

Symbol (:user) tekil bir nesnedir; ayni sembol her zaman ayni object_id'ye sahiptir. String her olusturulduygunda yeni bir nesne yaratir. Semboller genelde hash key ve sabit isimler icin kullanilir.

3 attr_accessor, attr_reader, attr_writer farklari?

attr_reader: sadece okuma (getter). attr_writer: sadece yazma (setter). attr_accessor: hem getter hem setter olusturur. Ornek: attr_accessor :name ile hem name hem name= kullanilabilir.

4 Proc ile Lambda arasindaki arguman farki?

Proc fazla veya eksik argumanda hata vermez (eksigi nil sayar, fazlasini yok sayar). Lambda arguman sayisina katidir; yanlis sayida arguman verilirse ArgumentError olur.

5 ? ve ! ile biten metotlari anlami?

? ile biten metotlar boolean doner (even?, zero?). ! ile biten metotlar destructive/tehlikeli islem yapar; orijinal veriyi degistirir (upcase! gibi).

6 ORM'in avantajlari nelerdir?

Daha az SQL yazilir, nesne yonelimli calisilir, veritabani platformundan bagimsizlik saglanir (PostgreSQL'den MySQL'e gecis kolay), modelleme uygulama tarafinda yapilir, bakim maliyeti dusuktur.

7 find, find_by ve where arasindaki fark?

find(id): id ile arar, bulamazsa exception firlatir. find_by(...): tek kayit veya nil doner. where(...): sifir veya cok kayit donen ActiveRecord::Relation nesnesi doner.

8 save ile create arasindaki fark?

Post.new ile once nesne olusturulur, save ile kaydedilir (iki adim). Post.create ile nesne olusturulur ve aninda veritabanina yazilir (tek adim).

9 db:setup ile db:reset ne yapar?

db:setup = schema:load + seed (sema yuklenir, seed verisi eklenir). db:reset = db:drop + setup (veritabani silinir, sifirdan olusturulur ve seed eklenir).

10 Migration rollback nasil yapilir?

rails db:rollback komutu son migration'i geri alir. rails db:migrate:status ile durum kontrol edilir. STEP parametresi ile birden fazla migration geri alınabilir.

11 GET ve POST ne zaman kullanilir?

GET: veri okumak icin (sayfa gosterme, listeleme). POST: yeni kayıt olusturmak icin (form gonderme). GET ile hassas veri gonderilmemeli.

12 PUT ile PATCH farki?

PUT kaynagin tum alanlarini gunceller. PATCH sadece degisen alanlari gunceller (kismi guncelleme). Ikisi de mevcut kaydi guncellemek icin kullanilir.

13 resources :posts kac route olusturur? Isimleri?

7 CRUD route: index, show, new, create, edit, update, destroy. Ornek: GET /posts (index), GET /posts/:id (show), POST /posts (create), DELETE /posts/:id (destroy).

14 posts_path ile posts_url farki?

posts_path sadece yol doner: /posts. posts_url tam URL doner: http://localhost:3000/posts (protokol + host + port dahil).

15 Enum'da draft: 0 ne anlama gelir?

Veritabaninda status alanı 0 olarak saklanır, Ruby tarafında draft anlamına gelir. Yeni kayıt default 0 ise otomatik draft olur. post.draft? ve Post.draft scope kullanılabilir.

16 Strong Parameters neden gerekli?

Mass assignment saldırısını onler. Sadece izin verilen alanlar (permit) modele aktarılır. Örnek: `params.require(:post).permit(:title, :article)` — baska alanlar yok sayılır.

17 authenticity_token ne ise yarar?

CSRF (Cross-Site Request Forgery) koruması sağlar. Formda hidden field olarak gönderilir, Rails oturumdaki token ile karşılaştırır. Eşleşmezse istek reddedilir.

18 Partial dosya adlandırma kuralı?

Partial dosya adı `_` ile başlar: `_form.html.erb`. `render 'form'` veya `render partial: 'form'` ile çağrılır. Aynı formu new ve edit sayfalarında tekrar kullanmak için idealdir.

19 button_to ile link_to farkı (DELETE için)?

`link_to` sadece link oluşturur; DELETE için ek ayar gerekir. `button_to` kendi mini formunu oluşturur, `_method=delete` hidden field ekler — silme işlemi için daha güvenli ve standart.

20 before_action ne sağlar?

DRY prensibi: tekrarlayan kodu action'lardan önce çalıştırır. Örnek: `before_action :set_post, only: [:show, :edit, :update]` — `@post` her action'da otomatik yüklenir.

21 Bootstrap grid kaç sütundur?

12 sütunludur. Örnek: `col-4 + col-4 + col-4 = 12`, yan yana 3 eşit sütun. `col-8 + col-4 = 12`, geniş + dar sütun.

22 .container ile .container-fluid farkı?

`container`: ekran genişliğine göre max-width sınırlı (ortalanmış). `container-fluid`: her zaman %100 genişlik, sınır yok.

23 col-md-8 ne zaman yan yana, ne zaman alt alta?

768px ve üzeri ekranlarda yan yana durur. 768px altında (mobil) sütunlar otomatik üst üste yığılır (responsive).

24 belongs_to FK hangi tabloda?

Foreign key, belongs_to tanımlanan modelin tablosundadır. Örnek: Book belongs_to :author → books tablosunda author_id sütunu vardır.

25 has_many :through ne zaman kullanılır?

İki model arasında ara (join) tablo olduğunda. Örnek: Physician has_many :patients, through: :appointments — randevu tablosu üzerinden doktor-hasta ilişkisi.

26 HABTM join table özelliği?

id sütunu yoktur (id: false). Sadece iki foreign key içerir (post_id, category_id). Post ve Category çoktan çoklu bağlanır. Örnek: categories_posts tablosu.

27 dependent: :destroy ne yapar?

Üst kayıt silindiğinde bağlı alt kayıtları da otomatik siler. Örnek: Author silince has_many :books, dependent: :destroy ile kitapları da silinir.

28 Validation ile Normalization farkı?

Validation veriyi kontrol eder, geçersizse reddeder (save başarısız). Normalization veriyi kaydetmeden önce değiştirir (squish, titlecase) — reddetmez, düzenler.

29 create! ile create farkı?

create başarısız olursa false döner veya hatalı nesne döner. create! başarısız olursa exception fırlatır (ActiveRecord::RecordInvalid). ! isareti exception garantisi verir.

30 validates :title, presence: true, uniqueness: true, length: { maximum: 255 } ne kontrol eder?

Başlık boş olamaz, benzersiz olmalı (aynı başlıkla ikinci kayıt olamaz) ve en fazla 255 karakter olabilir.

31 HTTP 422 ne anlama gelir?

Unprocessable Entity — sunucu isteği anladı ama validasyon hataları nedeniyle işleyemedi. render :new, status: :unprocessable_entity ile form hataları gösterilir.

32 l18n.t ve l18n.l farkı?

`l18n.t` (translate): metin çevirir — buton, label, mesaj. `l18n.l` (localize): tarih ve saat nesnelerini yerel formata çevirir. Örnek: `t('actions.show')`, `l(Time.now)`.

33 FriendlyId ne sağlar?

URL'de id yerine okunabilir slug kullanır. `/posts/1` yerine `/posts/ruby-programlamaya-giris`. SEO için faydalıdır. `friendly_id :title, use: :slugged`

34 Action Text ne için kullanılır?

Zengin metin editörü (WYSIWYG) sağlar — kalın, italik, liste, resim ekleme. `has_rich_text :article` ve `form.rich_textarea :article` ile kullanılır.

35 Authentication ile Authorization farkı?

Authentication (kimlik doğrulama): Kullanıcı kim? — giriş yapma. Authorization (yetkilendirme): Ne yapabilir? — giriş sonrası erişim hakkı. Devise authentication, Pundit/CanCanCan authorization için.

36 `current_user` ve `user_signed_in?` ne döner?

`current_user`: giriş yapmış User nesnesini döner (yoksa nil). `user_signed_in?`: boolean — true ise oturum açık, false ise açık değil.

37 `authenticate_user!` ne yapar?

Giris yapılmamışsa kullanıcıyı login sayfasına yönlendirir. `before_action :authenticate_user!` ile korumalı sayfalar oluşturulur.

38 Admin namespace'te yetki kontrolü nasıl yapılır?

`before_action :authorize_admin` tanımlanır. `unless current_user.admin?` ise `root_path`'e redirect edilir. Sadece admin rolundekiler `admin/users` sayfalarına erişebilir.

39 `current_user.posts.new` neden kullanılır?

Post otomatik olarak giriş yapan kullanıcıya atanır. `user_id` elle göndermek yerine ilişki üzerinden oluşturma güvenli ve doğru yöntemdir.

Komut Hizli Referans

Terminalde en cok kullanacagin Rails komutlari. Ezber listesi olarak kullan.

BASH

```
rails new app -d postgresql --css bootstrap
rails s / rails c
rails g model Post title:text
rails g migration AddStatusToPost status:integer
rails db:create / db:migrate / db:rollback / db:seed / db:reset
rails routes
rails active_storage:install
bin/rails action_text:install
bundle add devise
rails generate devise:install
git add . && git commit -m "mesaj" && git push
```